

FACULDADE DE ENGENHARIA DA UNIVERSIDADE DO PORTO

Enhancing Centralized Multi-Robot Path Planning and Coordination

Afonso Tomás de Magalhães Mateus



Mestrado em Engenharia Eletrotécnica e de Computadores

Supervisor: Prof. Pedro Luís Cerqueira Gomes da Costa

Second Supervisor: Prof. Diogo Miguel Rodrigues Matos

May 11, 2026

Abstract

This dissertation addresses the problem of centralized coordination of multiple autonomous mobile robots operating in shared environments. As robotic fleets become increasingly prevalent in industrial, logistics, and service applications, the ability to compute coordinated, collision-free trajectories under real-world constraints becomes a critical requirement. This challenge is commonly formalized as the Multi-Agent Path Finding (MAPF) problem, which seeks to determine feasible paths for multiple agents from given start locations to specified goal locations while avoiding conflicts over time.

MAPF is computationally hard in its general form, which has motivated the development of practical planning approaches that trade optimality for scalability and real-time performance. In centralized fleet coordination systems, priority-based planners are often adopted due to their predictable behavior and computational efficiency. In particular, the centralized coordination framework developed at the Institute for Systems and Computer Engineering, Technology and Science (INESC TEC), within the Centre for Robotics in Industry and Intelligent Systems (CRIIS) and the Industry and Innovation Laboratory (iiLab), relies on a sequential Time-Enhanced A* (TEA*) planner, which incorporates temporal reasoning and space-time reservations to ensure conflict-free execution. While well suited for real-world deployment, this sequential structure introduces a significant limitation: planning outcomes are highly sensitive to the chosen robot priority ordering, and exhaustive exploration of all orderings is infeasible due to factorial growth with fleet size.

The main motivation of this work is to reduce the sensitivity of sequential TEA*-based planning to robot ordering without compromising the existing centralized architecture. Rather than replacing the underlying planner, this dissertation explores the use of parallel computation to improve solution robustness within a fixed planning time budget. The proposed approach executes multiple TEA* instances concurrently, each operating on a different priority ordering of the same MAPF instance, and selects the most suitable solution among those evaluated.

The solution builds upon a centralized planning framework in which a Supervisor issues planning requests composed of a set of agents and their associated start and goal vertices. Within the Path Planner, heuristic functions generate diverse priority orderings that are processed independently in parallel threads. Although TEA* remains sequential within each thread, the overall planning process benefits from parallelism at the system level. Alternative execution strategies are also discussed, including early solution selection based on partial thread completion or fixed time budgets, enabling a tunable trade-off between planning latency and exploration depth.

Finally, the document outlines a structured work plan guiding the implementation and experimental evaluation of the proposed solution. Overall, this dissertation aims to contribute a practical and extensible approach for improving centralized multi-robot coordination by leveraging parallel planning to enhance robustness and solution quality in TEA*-based fleet coordination systems.

Contents

1	Baseline System Stabilization	1
1.1	Initial System Instability Issues	1
1.2	Temporal Analysis of the Coordination System	2
1.2.1	Redundant Planner Executions	3
1.2.2	Planning with Temporally Inconsistent Robot States	5
1.2.3	Proposed Batching Mechanism	7
1.2.4	Effect of the Batching Mechanism	8
1.3	Summary	10

List of Figures

1.1	Simplified representation of the temporal flow associated with message processing and planning operations within the coordination system.	2
1.2	Sequential processing of nearly simultaneous planning requests, resulting in redundant planner executions.	3
1.3	Sequential processing of robot state updates leading to replanning based on a temporally inconsistent set of robot states.	6
1.4	Proposed coordination architecture with batching mechanisms for planning requests and robot state updates.	7
1.5	Execution flow after introducing batching mechanisms for planning requests and robot state updates.	9

List of Acronyms

APF	<i>Artificial Potential Field</i>
CCBS	<i>Continuous-Time Conflict-Based Search</i>
CBS	<i>Conflict-Based Search</i>
CBSH	<i>Heuristic-Guided Conflict-Based Search</i>
CBSH-RTC	<i>Real-Time Conflict-Based Search with Heuristics</i>
CRIIS	<i>Centre for Robotics in Industry and Intelligent Systems</i>
ECBS	<i>Enhanced Conflict-Based Search</i>
INESC TEC	<i>Institute for Systems and Computer Engineering, Technology and Science</i>
MAPF	<i>Multi-Agent Path Finding</i>
NP-hard	<i>Nondeterministic Polynomial-time Hard</i>
ROS	<i>Robot Operating System</i>
SIPP	<i>Safe Interval Path Planning</i>
TEA*	<i>Time-Enhanced A-Star</i>
iiLab	<i>Industry and Innovation Laboratory</i>

Chapter 1

Baseline System Stabilization

Before introducing new prioritization heuristics and parallel planning strategies, it was first necessary to stabilize the baseline coordination pipeline. Preliminary real-time experiments revealed temporal inconsistencies in the interaction between the robots' Navigation Handlers, the Supervisor, and the Path Planner during planning and replanning operations. This chapter analyzes the root causes of these inconsistencies and describes the modifications introduced to mitigate them.

1.1 Initial System Instability Issues

During planning and replanning operations, robots occasionally received paths whose initial segments no longer matched their current physical positions. This mismatch caused the navigation controller to observe a large error between the robot's current pose and the first target segment of the newly received path. When this error exceeded the controller's admissible threshold, the robot could stop instead of continuing the planned motion, potentially compromising the consistency and safety of the overall coordination process.

A preliminary analysis revealed that these inconsistencies were strongly related to the way planning requests and robot state updates were processed by the Supervisor component. More specifically, these operations could be triggered using outdated or temporally inconsistent robot states.

This issue became particularly relevant because the planner may require a non-negligible amount of computation time, especially in large environments or high density multi-robot scenarios. Consequently, the discrepancy between the planner's internal representation of the system and the real robot states could further increase before the generated plan was finally applied.

These observations motivated a deeper analysis of the temporal behavior of the coordination pipeline, with particular focus on message propagation delays, ROS callback queue processing, and replanning triggering mechanisms.

1.2 Temporal Analysis of the Coordination System

To better understand the origin of the previously described inconsistencies, a temporal analysis of the coordination pipeline was conducted. Figure 1.1 presents a simplified representation of the most relevant events and temporal intervals involved in the interaction between the robots and the Coordination System.

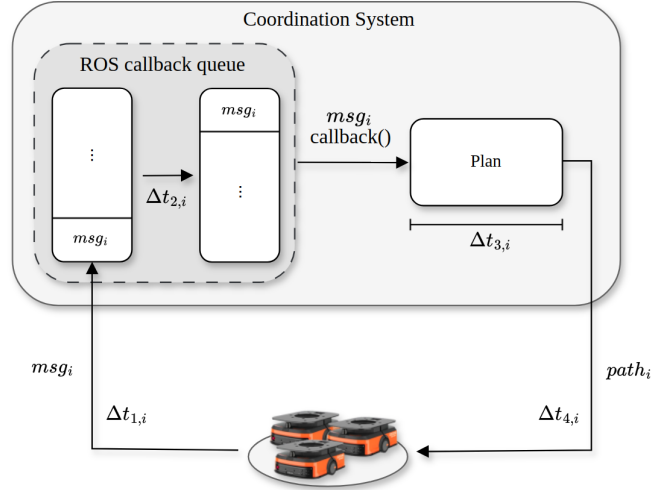


Figure 1.1: Simplified representation of the temporal flow associated with message processing and planning operations within the coordination system.

The presented abstraction focuses primarily on the temporal intervals associated with message transmission, ROS callback queue processing, and planner execution. In particular:

- $\Delta t_{1,i}$ corresponds to the transmission delay between robot i and the Supervisor. The transmitted messages may correspond to planning requests or robot state updates. Both types of messages are relevant in this context, since planning requests directly trigger planning operations, while robot state updates may trigger replanning operations when inconsistencies in the robots' progress are detected;
- $\Delta t_{2,i}$ represents the waiting time of incoming messages in the ROS callback queue before being processed by the Supervisor;
- $\Delta t_{3,i}$ denotes the planner execution time required to compute a new plan after a planning or replanning request is processed;
- $\Delta t_{4,i}$ corresponds to the transmission delay associated with sending newly computed plans from the Supervisor back to robot i .

Since the experiments were conducted in a fully local simulation environment, with all ROS nodes executing on the same machine, inter-process communication delays were expected to be negligible. As a result, the dominant source of temporal inconsistency was primarily associated

with the accumulation of delays introduced by sequential ROS callback processing and planner execution, corresponding mainly to the intervals $\Delta t_{2,i}$ and $\Delta t_{3,i}$. Therefore, the following subsections focus on these two temporal intervals, describing in detail the main issues identified in the baseline Coordination System.

1.2.1 Redundant Planner Executions

One of the main issues identified in the baseline coordination system was the redundant execution of the planner when multiple robots issued planning requests within a short time interval. In the coordination framework, each robot can individually request a plan to reach a given target vertex. However, in the original implementation, each planning request received by the Supervisor was immediately forwarded to the Path Planner, independently of whether requests from other robots had arrived at approximately the same time.

When a planning request was processed, the Supervisor invoked the Path Planner to compute a coordinated plan for the set of robots that required coordination at that instant. This set included the robots that had already submitted planning requests and had not yet reached their assigned destination. As a result, several planner executions could be triggered sequentially for what was effectively a single planning event, with all requests referring to the same global system state. More specifically, the first planner execution would only consider the request from the first robot processed by the Supervisor, the second would consider the requests from the first two robots, and this pattern would continue as additional callbacks were processed. Only the final planner execution would consider the complete set of planning requests submitted by the robots within that short time interval.

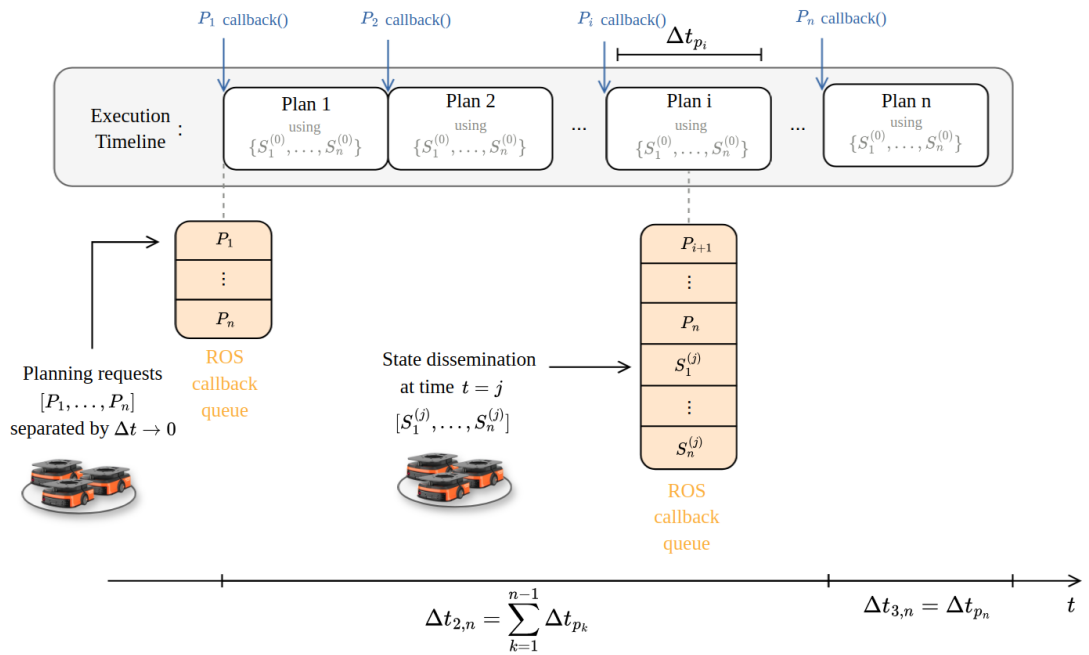


Figure 1.2: Sequential processing of nearly simultaneous planning requests, resulting in redundant planner executions.

As illustrated in Figure 1.2, the planning requests $\{P_1, \dots, P_n\}$ are sent by the robots within a very short time interval, represented by $\Delta t \rightarrow 0$. These requests are inserted sequentially into the ROS callback queue and processed one at a time by the Supervisor. In the baseline implementation, each processed request immediately triggers a new planner execution, leading to a sequence of n planning processes.

During this sequence, Plan 1 is computed after processing only P_1 , Plan 2 after processing $\{P_1, P_2\}$, and, more generally, Plan i after processing $\{P_1, \dots, P_i\}$. However, all these plans are computed using the same initial robot state set $\{S_1^{(0)}, \dots, S_n^{(0)}\}$, corresponding to the system configuration available before the first planning request was processed. Only Plan n considers the complete set of planning requests, $\{P_1, \dots, P_n\}$.

This behavior affected both computational efficiency and temporal consistency. The first $n - 1$ planner executions provided limited practical value, since each intermediate plan was immediately superseded by a later execution considering a larger and more complete set of robot requests. At the same time, these redundant executions contributed to blocking the ROS callback queue, delaying the processing of newly received robot state updates.

As also shown in Figure 1.2, a new state dissemination may occur at time $t = j$ during the execution of planning process i , producing updated robot states $\{S_1^{(j)}, \dots, S_n^{(j)}\}$. However, these state update messages are inserted after the pending planning requests in the ROS callback queue and therefore remain unprocessed until the previous planner executions have finished.

This is particularly problematic because the robots associated with the first $i - 1$ planning executions may already have received and started executing their newly computed paths. Consequently, their actual states at time $t = j$ may differ significantly from the states $S^{(0)}$ used when those paths were computed. Since the updated state messages remain blocked in the callback queue, this more recent information is only taken into account after the pending planner executions have finished.

In this scenario, the delay affecting the state information used by the final planning process can be approximated by the accumulated execution time of the previous planner calls. This corresponds to:

$$\Delta t_{2,n} = \sum_{k=1}^{n-1} \Delta t_{p_k} \quad (1.1)$$

, where Δt_{p_k} denotes the execution time of the k -th planning process. This accumulated delay is then followed by the computation time of the final planning process itself, given by:

$$\Delta t_{3,n} = \Delta t_{p_n} \quad (1.2)$$

It is also useful to consider the total response time between the moment a robot sends a planning request and the moment it receives the corresponding plan. Assuming that the message transmission delays $\Delta t_{1,n}$ and $\Delta t_{4,n}$ are negligible in the local simulation environment, this response time can be approximated as:

$$\Delta t_{\text{response},n} \approx \Delta t_{2,n} + \Delta t_{3,n} \quad (1.3)$$

For the final planning process, this results in:

$$\Delta t_{\text{response},n} \approx \sum_{k=1}^{n-1} \Delta t_{p_k} + \Delta t_{p_n} = \sum_{k=1}^n \Delta t_{p_k} \quad (1.4)$$

As a consequence, in addition to the significant increase in the total time required to generate a plan that considers all robots, the resulting plan may also be computed using states that no longer reflect the actual positions of the robots in the system.

1.2.2 Planning with Temporally Inconsistent Robot States

A second issue identified in the baseline coordination system was related to the way robot state updates were processed by the Supervisor. In the original implementation, whenever a robot sent a state update, the internal variables used to track that robot's progress were immediately updated. The updated progress was then compared with the progress of the remaining robots in order to determine whether a replanning operation should be triggered.

This behavior resulted from the coupling of two operations within the same ROS callback function, namely the update of the Supervisor's internal variables for the robot that sent the state message and the advancement of the state machines responsible for monitoring the execution progress of the robots. As a result, each individual state update could immediately influence the global replanning logic.

The progress of each robot was represented by its current position along the assigned path, typically expressed in terms of the number of completed path steps and the percentage of progression within the current step. If the difference between the progress of two or more robots exceeded a predefined threshold, the Supervisor interpreted this as a coordination inconsistency and triggered a new replanning operation.

However, this mechanism could lead to inconsistent replanning triggers because robot state updates were processed sequentially in the ROS callback queue. When the Supervisor processed the state update of a given robot, the states of the remaining robots might not yet have been updated with their most recent information. As a result, the progress comparison used to decide whether replanning was necessary could be based on a partially updated system state.

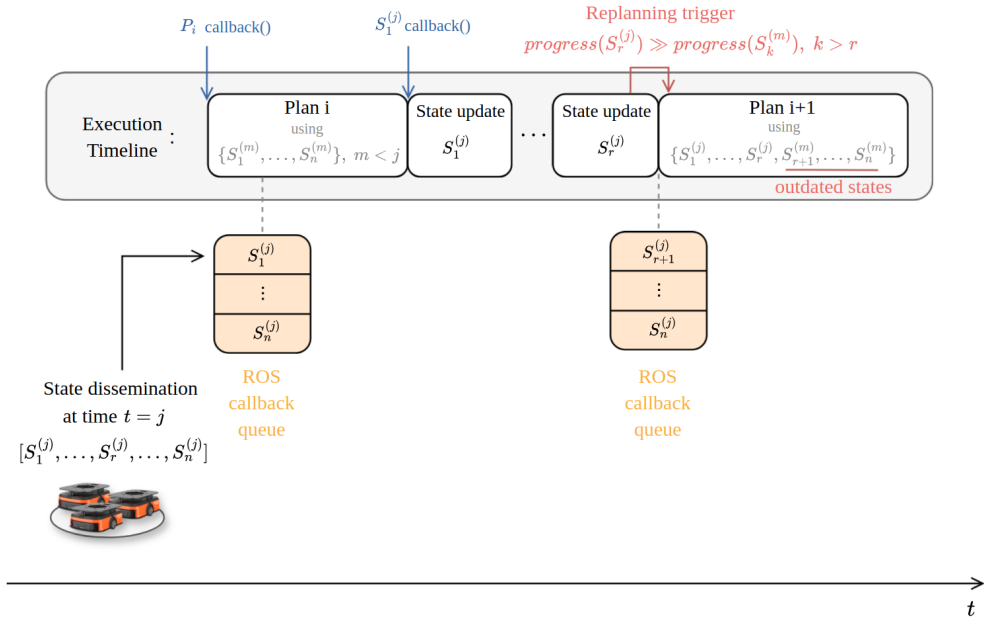


Figure 1.3: Sequential processing of robot state updates leading to replanning based on a temporally inconsistent set of robot states.

As illustrated in Figure 1.3, planning process i is initially executed using the state set $\{S_1^{(m)}, \dots, S_n^{(m)}\}$, corresponding to an earlier instant m . At a later instant $t = j$, a new state dissemination occurs, producing updated state messages $\{S_1^{(j)}, \dots, S_n^{(j)}\}$ for the robots. These messages are then inserted into the ROS callback queue and processed sequentially by the Supervisor.

For instance, after processing the callback associated with robot 1, the internal state of that robot is updated to $S_1^{(j)}$, while the states of the remaining robots may still correspond to older values. Later, when the callback associated with robot r is processed, the Supervisor evaluates the replanning condition by comparing the progress of robot r with the progress of the remaining robots.

This evaluation may trigger a replanning operation if the progress difference exceeds the predefined threshold, represented in the figure as $\text{progress}(S_r^{(j)}) \gg \text{progress}(S_k^{(m)})$, with $k > r$. However, at that moment, only part of the state dissemination has been processed. As a result, the new plan, denoted as Plan $i + 1$, may be computed using a mixed state set composed of updated states for the robots whose callbacks have already been processed, $\{S_1^{(j)}, \dots, S_r^{(j)}\}$, and outdated states for the remaining robots, $\{S_{r+1}^{(m)}, \dots, S_n^{(m)}\}$.

This behavior can introduce two related problems. First, the decision to trigger replanning could be affected by temporally inconsistent information, since the progress of one robot was compared against progress values from other robots that might still refer to older states. This could lead to unnecessary replanning operations.

Second, if replanning was triggered immediately after processing a single state update, the planner could be executed using robot states associated with different points in time. In this case,

the generated plan would not be based on a consistent global snapshot of the multi-robot system, but rather on a combination of updated and outdated agent states.

The main limitation of the baseline implementation was therefore not only the use of outdated state information, but the lack of temporal consistency among the robot states considered simultaneously by the Supervisor.

1.2.3 Proposed Batching Mechanism

To mitigate the temporal inconsistencies identified in the previous subsections, a batching mechanism was introduced for both planning requests and robot state updates. The main objective of this modification was to prevent the Supervisor from immediately triggering planning or replanning operations based on individual messages processed in isolation.

Instead of forwarding each planning request directly to the Path Planner, the system first aggregates requests received close in time. Similarly, robot state updates are collected and processed together before the Supervisor evaluates whether a replanning operation is required. This approach reduces redundant planner executions and allows replanning decisions to be based on a more temporally consistent representation of the multi-robot system.

The proposed architecture introduces two intermediate components between the robots and the Supervisor: the Robot Request Dispatcher and the Robot State Dispatcher.

The Robot Request Dispatcher is responsible for collecting planning requests from multiple robots and forwarding them to the Supervisor as a single batch. The Robot State Dispatcher performs an analogous role for robot state updates, ensuring that multiple state messages are processed together before progress comparisons are evaluated.

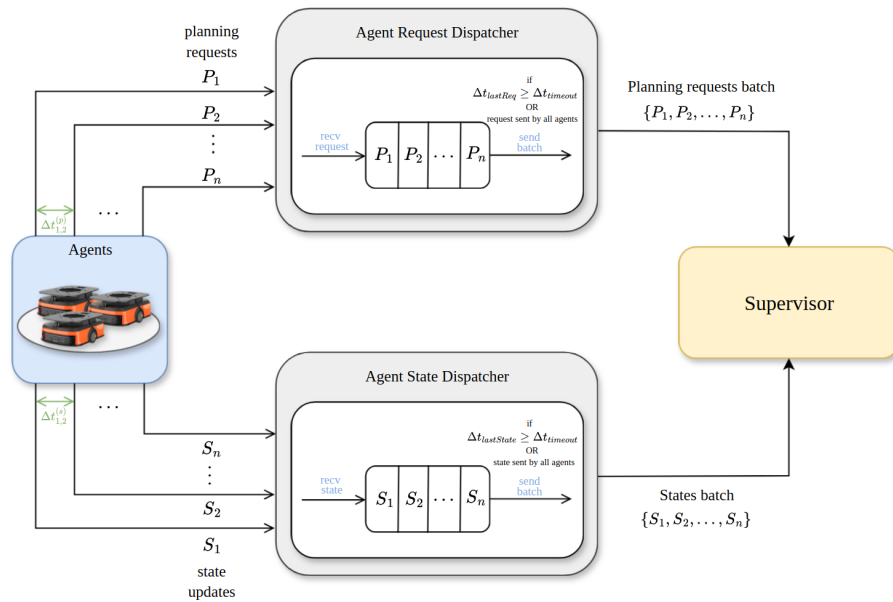


Figure 1.4: Proposed coordination architecture with batching mechanisms for planning requests and robot state updates.

As illustrated in Figure 1.4, planning requests are no longer processed independently as soon as they arrive at the Supervisor. Instead, the Robot Request Dispatcher accumulates incoming requests until one of two conditions is satisfied:

1. Planning requests have been received from all robots in the system, ensuring that the complete set of requests is available for the next planning operation;
2. A predefined inactivity timeout, denoted as T_{timeout} , expires after the most recent planning request, allowing requests generated close in time to be grouped together.

The timeout acts as an inactivity period that is restarted whenever a new planning request is received. Therefore, the batch is forwarded only when no new request has been received for the specified timeout duration, or when requests from all robots have already been collected. This mechanism prevents multiple nearly simultaneous requests from triggering multiple planner executions. As a result, a single planning process can be executed using the complete set of requests available for that planning event.

A similar strategy is applied to robot state updates through the Robot State Dispatcher. In contrast to the baseline implementation, where internal state updates and state-machine progression were performed within the same callback function, the proposed mechanism separates message reception from replanning evaluation. Multiple state messages are first grouped into a batch and only then forwarded to the Supervisor for processing.

The Supervisor can then update the internal states of all robots represented in the batch before advancing the corresponding state machines and comparing robot progress values. This ensures that replanning decisions are not triggered after an isolated state update, but rather after a more coherent set of robot states has been incorporated into the Supervisor's internal representation.

This modification directly addresses the problem of temporally inconsistent robot states. By processing state updates in batches, the Supervisor reduces the likelihood of evaluating replanning conditions using a mixture of recently updated and outdated states. Consequently, replanning decisions are based on a more coherent approximation of the global system state.

Overall, the batching mechanism introduces a controlled delay before processing planning requests and robot state updates. This delay is bounded by the timeout parameter, which must balance batch completeness and response latency. This bounded delay is preferable to the accumulation of redundant planner executions observed in the baseline implementation, while improving the temporal consistency of the planning pipeline.

1.2.4 Effect of the Batching Mechanism

The effect of the proposed batching mechanism is illustrated in Figure 1.5. In contrast to the baseline behavior, planning requests generated within a short time interval are first aggregated by the Robot Request Dispatcher before being forwarded to the Supervisor. As a result, the requests $\{P_1, \dots, P_n\}$ are processed as a single batch, producing only one planner execution instead of a sequence of redundant planning processes.

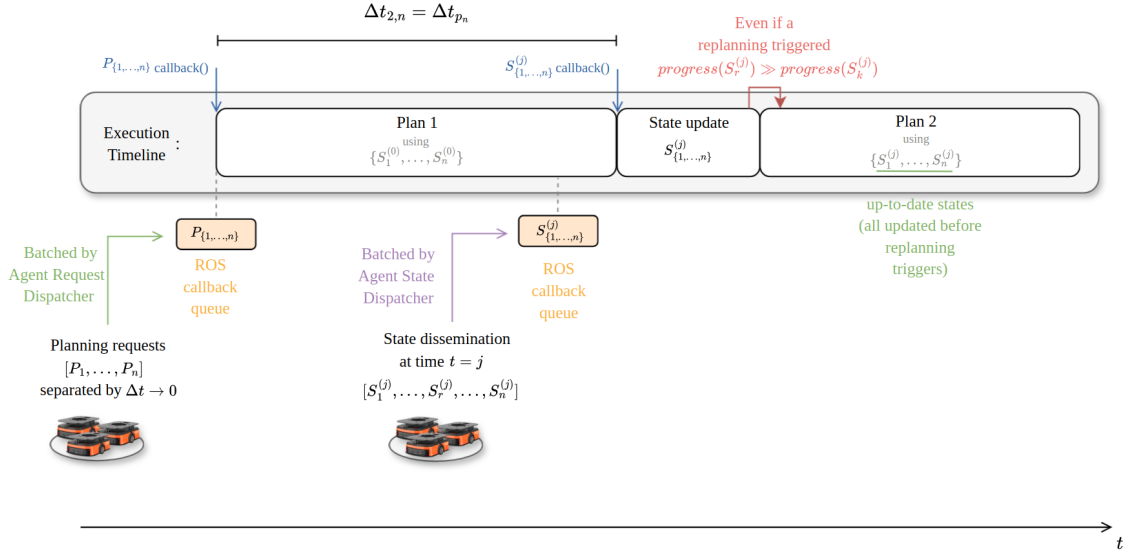


Figure 1.5: Execution flow after introducing batching mechanisms for planning requests and robot state updates.

As shown in the figure, the batched planning request triggers a single planning process, denoted as Plan 1, computed using the state set $\{S_1^{(0)}, \dots, S_n^{(0)}\}$. Since the requests $\{P_1, \dots, P_n\}$ are aggregated before being forwarded to the Supervisor, they are processed as a single planning event rather than as a sequence of independent callbacks. As a result, the $n - 1$ intermediate planner executions observed in the baseline implementation are avoided, and the waiting time associated with the ROS callback queue is significantly reduced. In this simplified representation, this waiting time is considered negligible:

$$\Delta t_2 \approx 0. \quad (1.5)$$

However, the batching mechanism introduces a new delay before the batch is forwarded to the Supervisor, denoted as Δt_{batch} . Since the inactivity timeout is restarted whenever a new request arrives, Δt_{batch} depends on the temporal distribution of incoming requests. In the worst case, if each new request arrives immediately before the inactivity timeout expires and the batch is completed only when the last robot sends its request, this delay may approach:

$$\Delta t_{\text{batch}} \approx (n - 1)T_{\text{timeout}}. \quad (1.6)$$

Nevertheless, T_{timeout} is selected to be significantly smaller than the planner execution time, making this additional batching delay negligible in relation to the computational cost of planning. After the batch is forwarded, the dominant remaining delay corresponds to the execution time of the single planning process:

$$\Delta t_3 = \Delta t_p. \quad (1.7)$$

Assuming that message transmission delays remain negligible in the local simulation environment, the total response time between the first planning request and the corresponding plan delivery can therefore be approximated as:

$$\Delta t_{\text{response}}^{\text{batch}} \approx \Delta t_{\text{batch}} + \Delta t_p. \quad (1.8)$$

This response time contrasts with the baseline behavior described in Equation 1.4, where the response time for the final planning process could accumulate the execution time of multiple redundant planner calls.

A similar improvement is obtained for robot state updates. When a new state dissemination occurs at time $t = j$, the updated states $\{S_1^{(j)}, \dots, S_n^{(j)}\}$ are first aggregated by the Robot State Dispatcher and then forwarded to the Supervisor as a single batch. This allows the Supervisor to update the internal state representation of the robots included in the batch before evaluating whether replanning is required.

With batching, state updates accumulated before the inactivity timeout expires are incorporated before the progress comparison is performed. Therefore, if replanning is triggered, the new plan is computed using a more coherent and up-to-date state set, $\{S_1^{(j)}, \dots, S_n^{(j)}\}$, rather than a state representation made inconsistent by partially processed callback messages.

This mechanism therefore introduces a trade-off between temporal consistency and response latency. Since messages are accumulated before being forwarded to the Supervisor, planning requests and robot state updates are no longer processed immediately after reception. In the considered system, this additional delay is not expected to compromise coordination performance, since robot states are published at a higher frequency than that required by the Supervisor to make coordination decisions. If response latency became critical, more recent robot states could be estimated through interpolation between consecutive updates. However, this mechanism was not required in the evaluated system.

1.3 Summary

This chapter presented the stabilization process applied to the baseline coordination pipeline. The main issues identified were related to redundant planner executions and replanning decisions based on temporally inconsistent robot states, both caused by the sequential processing of messages in the ROS callback queue. To mitigate these issues, batching mechanisms were introduced for planning requests and robot state updates. These mechanisms reduced unnecessary planner calls and allowed replanning decisions to be based on more coherent state snapshots. As a result, the baseline system became more temporally consistent and reliable, providing a stable foundation for the planner improvements developed in the following chapters.